

Integration du modele Hugging Face pour la generation automatique de panier

Document technique de synthese pour le rapport de donnees et la partie implementation. Il retrace le besoin, les choix d'architecture, le deploiement gratuit via Hugging Face Space, l'integration backend/frontend, les tests effectues et les limites observees.

Element	Valeur
Projet	SOUKI - Panier intelligent IA
Modele	TahaBDI/gemma-2-2b-panier-merged
Service applicatif	FastAPI backend + Next.js frontend
Deploiement gratuit retenu	Hugging Face Space Docker CPU Basic
Endpoint backend	POST /api/paniers/generer

1. Objectif de l'integration

L'objectif etait de remplacer un service ML de test par un service reel capable de generer un panier automatique a partir des choix de l'utilisateur: budget, nombre de personnes, duree et profil culinaire. Le resultat doit ensuite etre transforme en lignes panier exploitables par le checkout avec les produits, quantites, prix exacts et niveaux catalogue deja presents dans l'application.

- Utiliser le modele final merge depose sur Hugging Face.
- Transformer les criteres utilisateur en prompt envoye au modele.
- Parser une reponse JSON contenant une composition de produits.
- Mapper la sortie du modele vers les produits reels du catalogue SOUKI.
- Appliquer les prix backend exacts: prix_affiche puis prix_kg.
- Verifier stock, niveaux 1/2/3, quantites positives et persistance du panier.
- Envoyer l'utilisateur vers le checkout avec un panier_id fiable.

2. Architecture cible

L'architecture finale garde le backend comme source de verite. Le frontend ne calcule pas les prix et ne fait pas confiance directement au modele. Il envoie seulement les criteres du panier intelligent. Le backend interroge le modele, normalise la reponse, consulte la base de donnees et cree un panier brouillon.

Couche	Responsabilite
Frontend Next.js	Afficher la modale IA, collecter budget/personnes/duree/profil, appeler /api/paniers/generer.
Controller panier	Exposer POST /api/paniers/generer et recuperer l'utilisateur connecte.
MLPanierService	Construire le prompt, appeler HF/Space, parser JSON, appliquer les regles metier.
IPanierDao / PanierDaoBD	Lire les produits actifs, creer panier brouillon et lignes panier.
Checkout	Recharger le panier via panier_id et finaliser la commande.

3. Deploiement gratuit sur Hugging Face

Le serverless Hugging Face Inference n'a pas pu servir directement le repo modele: le provider a indique que le modele n'etait pas supporte. Pour rester en gratuit, l'option retenue est un Hugging Face Space Docker CPU Basic. Ce choix permet d'exposer une API HTTP compatible avec le backend, au prix d'un premier appel plus lent et d'une capacite CPU limitee.

3.1 Creation du Space

- Creer un Space: TahaBDI/souki-panier-api.
- Choisir SDK Docker.
- Choisir Hardware CPU Basic - Free.
- Ajouter les secrets HF_TOKEN et MODEL_REPO_ID.
- Garder le Space private si le modele doit rester protege.

3.2 Fichiers du Space

README.md

```
---
title: Souki Panier API
sdk: docker
app_port: 7860
models:
  - TahaBDI/gemma-2-2b-panier-merged
---
```

Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends build-essential \
    && rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 7860
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "7860"]
```

requirements.txt

```
fastapi==0.115.6
uvicorn[standard]==0.34.0
transformers==4.48.0
accelerate==1.2.1
torch==2.5.1
safetensors==0.4.5
pydantic==2.10.4
```

app.py - version API simplifiee

```
import json, os, re, torch
from typing import Any
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from transformers import AutoModelForCausalLM, AutoTokenizer

MODEL_REPO_ID = os.getenv("MODEL_REPO_ID", "TahaBDI/gemma-2-2b-panier-merged")
HF_TOKEN = os.getenv("HF_TOKEN")
app = FastAPI(title="Souki Panier API")
tokenizer = None
model = None

class GenerateRequest(BaseModel):
    inputs: str
    parameters: dict[str, Any] | None = None
    options: dict[str, Any] | None = None

def load_model():
    global tokenizer, model
    if tokenizer is not None and model is not None:
        return
    tokenizer = AutoTokenizer.from_pretrained(MODEL_REPO_ID, token=HF_TOKEN, trust_remote_code=True)
    model = AutoModelForCausalLM.from_pretrained(
        MODEL_REPO_ID, token=HF_TOKEN, torch_dtype=torch.float32,
        low_cpu_mem_usage=True, trust_remote_code=True
    )
    model.eval()

@app.get("/")
def health():
    return {"status": "ok", "model": MODEL_REPO_ID, "loaded": tokenizer is not None and model is not None}

@app.post("/")
def generate(payload: GenerateRequest):
    try:
        load_model()
        params = payload.parameters or {}
        inputs = tokenizer(payload.inputs, return_tensors="pt")
        with torch.inference_mode():
            output_ids = model.generate(
                **inputs,
                max_new_tokens=int(params.get("max_new_tokens", 512)),
                do_sample=bool(params.get("do_sample", False)),
                pad_token_id=tokenizer.eos_token_id,
            )
        generated = tokenizer.decode(output_ids[0], skip_special_tokens=True)
        if generated.startswith(payload.inputs):
            generated = generated[len(payload.inputs):].strip()
        return [{"generated_text": generated}]
    except Exception as exc:
        raise HTTPException(status_code=500, detail=str(exc))
```

4. Configuration backend SOUKI

Le backend est configure par le fichier back-end/.env. L'URL du Space remplace l'ancien endpoint serverless Hugging Face. Le token est necessaire si le Space est prive.

```
SOUKI_ML_INFERENCE_URL=https://tahabdi-souki-panier-api.hf.space/
HF_TOKEN=hf_XXXXXXXXXXXXXXXXXXXX
SOUKI_ML_PRELOAD_MODEL=1
SOUKI_ML_PRELOAD_TIMEOUT_SECONDS=180
```

Le chargement des variables d'environnement a ete ajuste afin que le fichier .env backend prenne bien le dessus sur une variable deja presente dans le terminal. Cela evite qu'une ancienne valeur comme SOUKI_ML_PRELOAD_MODEL=0 reste active malgre la modification du fichier.

5. Integration backend

- Ajout de panier_id dans PanierResponseDTO.
- Extension de IPanierDao avec get_active_products_for_ml.
- Implementation de get_active_products_for_ml dans PanierDaoBD.

- Injection de IPanierDao dans MLPanierService via dependencies.py.
- Suppression du couplage direct du service ML vers PanierDaoBD.
- Creation du panier brouillon et des lignes panier apres generation.
- Warmup remote optionnel au demarrage si SOUKI_ML_PRELOAD_MODEL=1.

5.1 Flux d'execution

Etape	Description
1	Le frontend envoie budget, personnes, duree et profil.
2	Le controller /api/paniers/generer recupere l'utilisateur connecte.
3	Le service lit les produits actifs en stock via IPanierDao.
4	Le prompt est construit avec les criteres et le catalogue.
5	Le Space HF retourne une composition JSON.
6	Le backend mappe chaque item vers Product et applique les prix exacts.
7	Le panier est complete pour respecter les niveaux 1, 2 et 3 si disponibles.
8	Le panier brouillon est cree en base et son panier_id est renvoye au frontend.

6. Integration frontend

- La modale IA appelle generateSmartPanier dans front-end/lib/catalogue.ts.
- Le bouton Aller au checkout transmet source=smart et panier_id.
- Le checkout recharge les lignes depuis /api/paniers/{panier_id}.
- Une interface de suggestions par niveau apparait sous le panier genere.
- Les profils du modele sont maintenant visibles sous forme de grille selectionnable.

Profil visible	Objectif
Equilibre	Panier varie pour usage general.
Legumes de base	Essentiels du quotidien.
Salade fraicheur	Produits frais et crudites.
Soupe hiver	Legumes adaptes aux soupes.
Cuisine tajine	Selection orientee plats marocains.
Cuisine couscous	Profil couscous complet.
Fruits dominant	Panier avec plus de fruits.
Legumes verts	Produits verts et legers.
Racines & tubercules	Produits comme pommes de terre et carottes.
Aromates & herbes	Menthe, persil, coriandre et aromates.

7. Tests et constats

- Verification des routes FastAPI: /api/paniers/generer est bien enregistree.
- Verification du frontend: la modale smart appelle bien /api/paniers/generer.
- Test du provider HF serverless: refus avec Model not supported by provider hf-inference.

- Correction vers Hugging Face Space gratuit comme endpoint externe.
- Verification TypeScript: tsc --noEmit OK.
- Verification Python: py_compile OK sur les fichiers backend modifies.
- Correction catalogue: Aubergines remises a 8 DH dans prix_kg, prix_gros_saisi et prix_affiche.

Point important

Si le Space est prive, l'ouverture directe de <https://tahabdi-souki-panier-api.hf.space/> dans le navigateur peut afficher 404. Ce comportement est normal. Le backend doit appeler le Space avec Authorization: Bearer HF_TOKEN.

8. Limites de l'option gratuite

- CPU Basic gratuit peut etre lent pour Gemma 2B.
- Le premier appel peut prendre plusieurs minutes car le Space charge le modele.
- Le Space peut dormir apres inactivite.
- Une erreur memoire est possible si le modele est trop lourd en float32.
- Pour production stable, un Inference Endpoint dedie reste plus robuste mais payant.

9. Checklist de reprise

Verification	Commande ou action
Space actif	Ouvrir le Space dans Hugging Face et verifier Running.
Health API	GET / avec token si Space prive.
Variables backend	Verifier SOUKI_ML_INFERENCE_URL, HF_TOKEN, SOUKI_ML_PRELOAD_MODEL.
Backend	Redemarrer uvicorn apres modification du .env.
Catalogue	Verifier Aubergines: prix_kg=8, prix_affiche=8.
Frontend	Ouvrir catalogue, Panier intelligent, choisir un profil visible.
Checkout	Confirmer que l'URL contient panier_id et que les lignes se rechargent.

Conclusion: le service SOUKI conserve la logique metier dans le backend, le modele HF sert uniquement a proposer une composition, et le checkout s'appuie sur les donnees catalogue fiables de la base. Cette separation limite les erreurs de prix, de stock et de produits inexistants.